

Final Project: Milestone 4

Project Timeline

Milestone	Due Date	Deliverable
4 — Distributed	May 27	Code + report with MPI implementation, benchmarks
Final Report	Jun 8	Code + 6-page final report

Milestone 4 Instructions

Overview

The goal of Milestone 4 is to scale your single-GPU implementation from Milestone 3 to a distributed multi-GPU setting using MPI. By the end of this milestone, you should have a working multi-process implementation in which each MPI rank manages one GPU, together with a study of how your code scales with the number of ranks.

The focus of this milestone is on the design and implementation of the distributed algorithm, the communication between ranks, and performance. You should think explicitly about how computation is partitioned across ranks, what data must be exchanged at each step, and how communication overlaps (or does not overlap) with computation. Whenever possible, you are encouraged to use MPI collective operations (MPI_Allreduce, MPI_Allgather, MPI_Bcast, etc.) rather than point-to-point patterns: collectives are better optimized for the underlying network topology and lead to cleaner, more portable code.

Computing Resources

The teaching cluster is a shared resource and the number of GPUs available at any given time is limited. Please plan your runs accordingly:

- Individual jobs submitted to the teaching cluster should not run for more than a few minutes, and **never more than 15 minutes**.
- Use small problem sizes for development and debugging, and reserve larger runs for the final benchmarking pass.
- If you have access to your own computing resources (lab cluster, cloud credits, personal multi-GPU workstation, etc.), you are welcome to use them for the final project. Please document the hardware you used in your report.
- Plan your scaling study carefully so that the total cluster time you use is reasonable.

Deliverables

You will turn in two items:

1. **Code.** The MPI + CUDA implementation of your algorithm, together with the C++ wrapper code (host-side driver and any CPU reference or baseline code) needed to build, run, and test it on multiple ranks.
2. **Progress report (approximately 2 pages, excluding references and figures).** A written description of your progress since Milestone 3, your current status, and a performance and scalability analysis of your distributed implementation.

Report Content

Your progress report should address the following:

1. **Summary of progress.** Briefly describe what you have implemented since Milestone 3. Which components of your design are complete in the distributed version, which are partially done, and which are not yet started? Note any changes to your design or to your single-GPU kernels that were required to support the distributed implementation, and explain why.
2. **Distributed algorithm and data decomposition.** Describe how the problem is decomposed across MPI ranks. What data does each rank own, and what does each rank need from its neighbors at each step of the algorithm? Discuss any alternative decompositions you considered (e.g., 1D vs. 2D partitioning, replicated vs. distributed parameters, data vs. model parallel) and explain the trade-offs that led to your choice.
3. **MPI implementation.** Describe the communication pattern of your implementation. Which MPI collectives are you using, and at which points in the algorithm? If you use point-to-point communication, justify the choice. Discuss how (or whether) you overlap communication with computation, and how GPU buffers interact with MPI (e.g., staging through host memory vs. CUDA-aware MPI).
4. **C++ wrapper and integration with CUDA.** Describe how your host code coordinates MPI calls and CUDA kernel launches: device selection per rank, memory management across host and device, and any changes needed to your Milestone 3 kernels.
5. **Correctness testing.** Describe how you verify that the distributed implementation produces results consistent with your single-GPU code (and, if applicable, your CPU reference). Test across different rank counts (e.g., 1, 2, 4 ranks). Report the accuracy achieved.
6. **Performance, scaling, and profiling.** Report performance measurements for your distributed implementation. At a minimum, include:
 - A *strong scaling* study (fixed total problem size, increasing rank count) and/or a *weak scaling* study (fixed problem size per rank, increasing rank count).
 - A breakdown of the time spent in computation vs. communication.
 - Speedup and parallel efficiency curves, with at least one figure or table.

Use a profiling tool for your analysis. Nsight Systems is useful for end-to-end timelines on distributed runs, and Nsight Compute for analyzing individual kernels. Identify the main scalability bottlenecks. Examples include communication volume, communication latency, load imbalance, kernel launch overhead, and host-device transfers.

7. **Discussion of approaches and bottlenecks.** Reflect on the design choices you made. What would you do differently if you had more time or more hardware? Which optimizations gave the largest improvements, and which were not worth their complexity? At what scale do you expect your current implementation to stop scaling, and why?
8. **Challenges and next steps.** List the tasks remaining for the final report, with an updated timeline.

Submission

Submit your code and your progress report on Gradescope as a group. The report should be approximately 2 pages (excluding references and figures), single-spaced, 11pt font, 1-inch margins. Include all group members in the submission.